

Πανεπιστήμιο Κρήτης, Τμήμα Επιστήμης Υπολογιστών
HY252 – Αντικειμενοστρεφής Προγραμματισμός
Εξάμηνο: Χειμερινό εξάμηνο 2025-2026
Διδάσκων: **Γιάννης Τζίτζικας**
Υπεύθυνος: Αλέξανδρος Μάνος (5136)

3η Σειρά Ασκήσεων

Ανάθεση: 12 Δεκεμβρίου 2025
Παράδοση: 12 Ιανουαρίου 2026
Αξία: 1,5% του τελικού βαθμού στο μάθημα

Εκπαιδευτικοί στόχοι

- *Γνωριμία και εξάσκηση με τις νέες λειτουργίες που προφέρει η Java 8 (interfaces, lambda expressions, streams). Εκτός των διαφανειών και του σχετικού κεφαλαίου του βιβλίου "Μια Γεύση από Java, θα χρειαστεί να μελετήσετε και μόνοι σας.*

Σημειώσεις

1. **Απορίες** σχετικά με την Α3 θα απαντώνται μόνο μέσω του forum της Α3 στο moodle.
2. **Οδηγίες Παράδοσης**. Δείτε τις οδηγίες που αναγράφονται σε ξεχωριστό αρχείο.
3. Παραβίαση των κανόνων παράδοσης συνεπάγεται είτε τη μη διόρθωση της εργασίας είτε 30% μείωση του βαθμού (αν διορθωθεί).

Η εργασία είναι ατομική και απαγορεύεται η χρήση κώδικα που δεν έχετε γράψει οι ίδιοι, είτε από το διαδίκτυο είτε από κάποιο συμφοιτητή σας. Σε περίπτωση εντοπισμού αντιγραφής η εργασία θα μηδενίζεται. Αν κάτι αντιγράψετε από κάπου (όσο μικρό ή μεγάλο είναι), πρέπει να το αναφέρετε στην αναφορά.



Άσκηση 1 (20%)

Java 8 interfaces

1.1 [5%] Έχετε το παρακάτω Interface που περιγράφει μια έξυπνη συσκευή και μια υλοποίηση. Ο παρακάτω κώδικας δεν γίνεται compile. Εμπλουτίστε **μόνο την SmartDevice** (χωρίς να πειράξετε την SmartBulb) ώστε να λυθεί το πρόβλημα.

```
interface SmartDevice
{
  public void turnOn();
  public void turnOff();
  public void reset();
```



```
}

class SmartBulb implements SmartDevice{
  public void turnOn() { System.out.println("Bulb is ON"); }
  public void turnOff() { System.out.println("Bulb is OFF"); }
}
```

1.2 [5%] Κάντε το ίδιο και για το νέο interface NetworkComponent που ακολουθεί, και το οποίο δηλώνεται ότι υλοποιείται από την SmartBulb. Στη συνέχεια εμπλουτίστε το interface NetworkComponent με μία μέθοδο public void reset(). Λύσετε το πρόβλημα σύγκρουσης που θα προκύψει στην SmartBulb λαμβάνοντας υπόψη ότι η SmartBulb θα θέλαμε να κληρονομεί την υλοποίηση της reset() της διεπαφής NetworkComponent.

```
interface NetworkComponent
{
  public void connect();
}


class SmartBulb implements SmartDevice, NetworkComponent{
}
```

1.3 [5%]
Προσθέστε στο interface **SmartDevice** μια στατική μέθοδο:

```
static boolean isDeviceOnline(String status)
{
  return "ONLINE".equals(status);
}
```

Στη main σας, καλέστε αυτή τη μέθοδο για να ελέγξετε ένα string (π.χ. "ONLINE") και τυπώστε το αποτέλεσμα. Εξηγήστε σύντομα σε σχόλιο γιατί αυτή η μέθοδος δεν κληρονομείται από την κλάση SmartBulb (δηλαδή γιατί το bulb.isDeviceOnline(...) δεν λειτουργεί).

1.4 [5%] Ο παρακάτω κώδικας δεν γίνεται compile. Εξηγήστε το πρόβλημα και προτείνετε πιθανές λύσεις.

```
import java.util.function.Predicate;
interface LongPredicate
{
   public boolean test(int v);
}
```

```
public class ExerciseAmbiguity {
 private static void method(Predicate<Long> p)
 {
   System.out.print("Predicate");
 }

private static void method(LongPredicate p)
 {
     System.out.print("IntPredicate");
 }

public static void main(String args[])
 {
   method((x) -> true);
 }
}
```



Άσκηση 2 (40%)

Lambdas and streams

2.1 [10%] Γράψτε πιο σύντομα τον παρακάτω κώδικα χρησιμοποιώντας τις δυνατότητες της Java 8:

```
class A21 {
  public static void main(String[] aa) {
    JFrame frame = new JFrame();
    JButton button1 = new JButton("Click me!");
    button1.addActionListener(new ActionListener(){
          public void actionPerformed(ActionEvent e)
          {
            System.out.println("Hello Java 8!");
            Runnable r1 = new Runnable() {
                @Override
                public void run() {
                  System.out.println("Τρέχω παράλληλα");
                  main(null);
                }
              };
            new Thread(r1).start();
          }
     });
    frame.add(button1);
```

```
    frame.setSize(200, 200);
    frame.setVisible(true);
  } // main
}
```

2.2 [15%] Γράψτε μια πιο σύντομη ισοδύναμη έκδοση, του κώδικα μεταξύ start και end χρησιμοποιώντας τις δυνατότητες της Java 8, συγκεκριμένα τα streams της Java 8. Ο νέος κώδικας πρέπει να δουλεύει σωστά ακόμα και αν το ArrayList είναι άδειο. Επίσης ο υπολογισμός του γινομένου πρέπει να γίνει με την εντολή **reduce**.

```
class a23_ekfonisi {
      public static void main(String[] args) {
        ArrayList<Double> items = new ArrayList<>();
        for (int i=0; i < 30; i++)
            items.add(i*i/3.3);
        // Start
        double max = Double.NEGATIVE_INFINITY;
        double min = Double.POSITIVE_INFINITY;
        double sum = 0.0;
        double product = 1.0;
        for (Double item : items) {
            if (item>max) max = item;
            if (item<min) min = item;
            sum += item;
            product *= item;
        }
        System.out.println("Minimum = " + min);
        System.out.println("Maximum = " + max);
        System.out.println("Average = " + sum/(double)items.size());
        System.out.println("Product = " + product);
       //end
}}
```

2.3 [15%] Ο παρακάτω κώδικας αφού δημιουργήσει μια λίστα από 20 τυχαίους αριθμούς από το 1 έως το 100, εκτυπώνει στοιχεία της λίστας βάσει κάποιων κριτηρίων.

```
class a24 {
      public static void main(String[] a) {
            final int K = 20 ;
        ArrayList<Integer> items = new ArrayList<>();
        System.out.println("Initial list: ");
        for (int i = 0; i < K; i++) {
            items.add(new Random().nextInt(100)+1);
            System.out.print(items.get(i) + " ");
        }
        System.out.print("\n");
```

```
        // start
        int val1 = 15;
        System.out.println("Numbers bigger than " + val1 +":");
        for (int i = 0; i < K; i++) {
            if (items.get(i) > val1) {
                System.out.print(items.get(i) + " ");
            }
        }
        System.out.print("\n");

        System.out.println("Numbers smaller than " + val1  +":");
        for (int i = 0; i < K; i++) {
            if (items.get(i) < val1) {
                System.out.print(items.get(i) + " ");
            }
        }
        System.out.print("\n");
        System.out.println("Numbers equal to 2 or between 10 and 20: ");
        for (int i = 0; i < K; i++) {
            if ((items.get(i) >=10 && items.get(i) <= 20) || items.get(i)==2) {
                System.out.print(items.get(i) + " ");
            }
        }
        System.out.print("\n");
        // end
}}
```

(α) Φτιάξτε μια στατική μέθοδο calculate η οποία να παίρνει ως παράμετρο μία λίστα ακεραίων και ένα predicate ακεραίων. Να εφαρμόζει το predicate και να εκτυπώνει στα στοιχεία που ικανοποιούν το predicate.

(β) Προσθέστε κώδικα στη main που να είναι ισοδύναμος με τον κώδικα από το start έως το end ο οποίος να αξιοποιεί τη μέθοδο που φτιάξατε στο (α), δηλαδή να την καλεί με παράμετρο την κατάλληλη έκφραση λάμδα.



Άσκηση 3 (20%)

Lambdas and streams (querying and analytics)

Ο παρακάτω κώδικας δημιουργεί μία κλάση που αναπαριστά ταινίες, ενώ η main έχει μία λίστα από κάποιες γνωστές ταινίες των τελευταίων (και όχι μόνο) χρόνων.

```
class Movie implements Comparable {

    private String title;
    private String type;
    private String director;

    private int year;

    private double rating;

    public Movie(String title, String type, String director, int year, double
rating) {
        this.title = title;
        this.type = type;
        this.director = director;
        this.year = year;
        this.rating = rating;
    }

    public int compareTo(Object carObj) {
        Movie movie = (Movie) carObj;
        return Double.compare(movie.rating, this.rating);
    }

    interface MovieFactory<P extends Movie> {
        P create(String title, String type, String director, int year, double
rating);
    }

    <… setters/getters …>

    public String toString() {
        return "[Title: " + title + " Type: " + type + " Director: " + director + "
Year: " + year + " Rating: " + rating + "]";
    }
}

public class a3{
```

```
      public static void main(String[] args) {
        Movie.MovieFactory<Movie> mv = Movie::new;
        ArrayList<Movie> movies = new ArrayList<>();

        movies.add(mv.create("2001: A Space Odyssey", "Science Fiction", "Stanley
Kubrick", 1968, 8.3));
        movies.add(mv.create("The Godfather", "Drama", "Francis Coppola", 1972,
9.2));
        movies.add(mv.create("Mulholland Drive", "Drama", "David Lynch", 2001,
7.9));
        movies.add(mv.create("Dune", "Science Fiction", "Denis Villeneuve", 2021,
8.2));
        movies.add(mv.create("Blade Runner 2049", "Science Fiction", "Denis
Villeneuve", 2017, 8.0));
        movies.add(mv.create("The Shining", "Horror", "Stanley Kubrick", 1980,
8.4));
        movies.add(mv.create("Apocalypse Now", "Drama", "Francis Coppola", 1968,
8.4));
        movies.add(mv.create("Ready Player One", "Science Fiction", "Steven
Spielberg", 2018, 7.4));
        movies.add(mv.create("Minority Report", "Action", "Steven Spielberg", 2002,
7.6));

      }
}
```

Συμπληρώστε **ΜΟΝΟ** τους απαραίτητους setters/getters.
Απαντήστε μόνο με χρήση streams/lambdas τα παρακάτω ερωτήματα:

3.1 Τυπώστε όλες τις ταινίες που έχει σκηνοθετήσει ο Stanley Kubrick
3.2 Τυπώστε όλες τις ταινίες που έχει σκηνοθετήσει ο Francis Coppola ή ο Denis Villeneuve
3.3 Τυπώστε όλες τις ταινίες που έχει σκηνοθετήσει ο ο Francis Coppola ή ο Denis Villeneuve ταξινομημένες βάσει της βαθμολογίας τους
3.4 Τυπώστε όλες τις ταινίες που έχει σκηνοθετήσει ο ο Francis Coppola ή ο Denis Villeneuve ταξινομημένες χρονολογικά
3.5 Τυπώστε όλες τις ταινίες που έχει σκηνοθετήσει ο ο Francis Coppola ή ο Denis Villeneuve ταξινομημένες χρονολογικά σε φθίνουσα σειρά
3.6 Τυπώστε την ταινία με την καλύτερη βαθμολογία (και δείτε την ☺)
3.7 Τυπώστε την ταινία που έχει σκηνοθετήσει ο Steven Spielberg με την καλύτερη βαθμολογία
3.8 Τυπώστε τις τρεις (3) καλύτερες ταινίες της λίστας βάσει της βαθμολογίας τους
3.9 Τυπώστε τον αριθμό των ταινιών που βγήκαν κάθε χρόνο
3.10 Τυπώστε τον μέσο όρο της βαθμολογίας των ταινιών ανά έτος

Ενδεικτικό output:

```
3.1 Movies Directed by Stanley Kubrick
[Title: 2001: A Space Odyssey Type: Science Fiction Director: Stanley Kubrick Year: 1968 Rating: 8.3]
[Title: The Shining Type: Horror Director: Stanley Kubrick Year: 1980 Rating: 8.4]

3.2 Movies Directed by Francis Coppola or Denis Villeneuve
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]
[Title: Dune Type: Science Fiction Director: Denis Villeneuve Year: 2021 Rating: 8.2]
[Title: Blade Runner 2049 Type: Science Fiction Director: Denis Villeneuve Year: 2017 Rating: 8.0]
[Title: Apocalypse Now Type: Drama Director: Francis Coppola Year: 1968 Rating: 8.4]

3.3 Movies Directed by Francis Coppola or Denis Villeneuve sorted by rating
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]
[Title: Apocalypse Now Type: Drama Director: Francis Coppola Year: 1968 Rating: 8.4]
[Title: Dune Type: Science Fiction Director: Denis Villeneuve Year: 2021 Rating: 8.2]
[Title: Blade Runner 2049 Type: Science Fiction Director: Denis Villeneuve Year: 2017 Rating: 8.0]

3.4 Movies Directed by Francis Coppola or Denis Villeneuve sorted by year
[Title: Apocalypse Now Type: Drama Director: Francis Coppola Year: 1968 Rating: 8.4]
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]
[Title: Blade Runner 2049 Type: Science Fiction Director: Denis Villeneuve Year: 2017 Rating: 8.0]
[Title: Dune Type: Science Fiction Director: Denis Villeneuve Year: 2021 Rating: 8.2]

3.5 Movies Directed by Francis Coppola or Denis Villeneuve sorted by year descending
[Title: Dune Type: Science Fiction Director: Denis Villeneuve Year: 2021 Rating: 8.2]
[Title: Blade Runner 2049 Type: Science Fiction Director: Denis Villeneuve Year: 2017 Rating: 8.0]
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]
[Title: Apocalypse Now Type: Drama Director: Francis Coppola Year: 1968 Rating: 8.4]

3.6 The best rated movie
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]

3.7 The best rated movie of Steven Spielberg
[Title: Minority Report Type: Action Director: Steven Spielberg Year: 2002 Rating: 7.6]

3.8: The three movies with the best rating
[Title: The Godfather Type: Drama Director: Francis Coppola Year: 1972 Rating: 9.2]
[Title: The Shining Type: Horror Director: Stanley Kubrick Year: 1980 Rating: 8.4]
[Title: Apocalypse Now Type: Drama Director: Francis Coppola Year: 1968 Rating: 8.4]

3.9: Number of movies by  year
{1968=2, 2017=1, 2001=1, 2002=1, 2018=1, 1972=1, 2021=1, 1980=1}

3.10: Average rating of movies by year
{1968=8.350000000000001, 2017=8.0, 2001=7.9, 2002=7.6, 2018=7.4, 1972=9.2, 2021=8.2, 1980=8.4}
```



Άσκηση 4 (20%)

Lambdas and streams (querying and analytics)

Όμοια με την Άσκηση 3, ο παρακάτω κώδικας δημιουργεί μία κλάση που αναπαριστά Laptops, ενώ η main έχει μία λίστα με διάφορα μοντέλα της αγοράς.

```
class Laptop {

    private String brand;
    private String model;
    private String cpu;

    private int ram;
    private double price;

    public Laptop(String brand, String model, String cpu, int ram, double price) {
        this.brand = brand;
        this.model = model;
        this.cpu = cpu;
        this.ram = ram;
        this.price = price;
    }

    interface LaptopFactory<P extends Laptop> {
        P create(String brand, String model, String cpu, int ram, double price);
    }

    <… setters/getters …>

    public String toString() {
        return String.format("[%s %s, CPU: %s, RAM: %dGB, Price: %.2f€]", brand,
model, cpu, ram, price);
    }
}

public class a4{

      public static void main(String[] args) {
        Laptop.LaptopFactory<Laptop> factory = Laptop::new;
        ArrayList<Laptop> laptops = new ArrayList<>();

       laptops.add(factory.create("Apple", "MacBook Air M2", "M2", 8, 1199.0));
       laptops.add(factory.create("Dell", "XPS 13", "i7-1250U", 16, 1450.0));
       laptops.add(factory.create("Lenovo", "ThinkPad X1 Carbon", "i7-1260P", 16, 1899.0));
```

```
        laptops.add(factory.create("Asus", "ZenBook 14", "Ryzen 7 5800H", 16, 999.0));
        laptops.add(factory.create("HP", "Pavilion 15", "i5-1135G7", 8, 650.0));
        laptops.add(factory.create("Apple", "MacBook Pro 16", "M2 Max", 32, 3200.0));
        laptops.add(factory.create("Dell", "Inspiron 15", "i3-1115G4", 8, 450.0));
        laptops.add(factory.create("Razer", "Blade 15", "i9-12900H", 32, 3600.0));
        laptops.add(factory.create("Lenovo", "IdeaPad 3", "Ryzen 5 5625U", 8, 520.0));
        System.out.println();
        // 4.1
        System.out.println("4.1 Laptops with >= 16GB RAM AND Price <= 1500€ ");

        // 4.2
        System.out.println("\n4.2 Unique Brands ");

        // 4.3
        System.out.println("\n4.3 Sorted by Price (Ascending) ");

        // 4.4
        System.out.println("\n4.4 Is there a cheap laptop (<400€)? ");

        // 4.5
        System.out.println("\n4.5 All models as one String ");

      }
}
```

Συμπληρώστε **MONO** τους απαραίτητους setters/getters.
Απαντήστε μόνο με χρήση streams/lambdas τα παρακάτω ερωτήματα:

4.1 Filter & Print: Τυπώστε όλα τα Laptops που έχουν μνήμη RAM μεγαλύτερη ή ίση με 16GB και ταυτόχρονα Τιμή μικρότερη ή ίση με 1500€.
4.2 Distinct: Τυπώστε τις μάρκες (Brands) που υπάρχουν στη λίστα, χωρίς διπλότυπα (π.χ. να φανεί το "Apple" μία φορά).
4.3 Sorting: Τυπώστε τα Laptops ταξινομημένα κατά Τιμή (αύξουσα σειρά).
4.4 AnyMatch: Ελέγξτε και τυπώστε (true/false) αν υπάρχει έστω και ένα Laptop στη λίστα με τιμή κάτω από 400€.
4.5 Joining: Φτιάξτε και τυπώστε ένα ενιαίο String που να περιέχει όλα τα ονόματα των μοντέλων χωρισμένα με κόμμα (π.χ. "MacBook Air M2, XPS 13, ...").
(hint: **joining** - Concatenate the input elements into a String, in encounter order. It is used inside `collect()` and can optionally take a delimiter, a prefix, and a suffix)

Ενδεικτικό output:

```
4.1 Laptops with >= 16GB RAM AND Price <= 1500€
[Dell XPS 13, CPU: i7-1250U, RAM: 16GB, Price: 1450.00€]
[Asus ZenBook 14, CPU: Ryzen 7 5800H, RAM: 16GB, Price: 999.00€]

4.2 Unique Brands
Apple
Dell
Lenovo
Asus
HP
Razer

4.3 Sorted by Price (Ascending)
[Dell Inspiron 15, CPU: i3-1115G4, RAM: 8GB, Price: 450.00€]
[Lenovo IdeaPad 3, CPU: Ryzen 5 5625U, RAM: 8GB, Price: 520.00€]
[HP Pavilion 15, CPU: i5-1135G7, RAM: 8GB, Price: 650.00€]
[Asus ZenBook 14, CPU: Ryzen 7 5800H, RAM: 16GB, Price: 999.00€]
[Apple MacBook Air M2, CPU: M2, RAM: 8GB, Price: 1199.00€]
[Dell XPS 13, CPU: i7-1250U, RAM: 16GB, Price: 1450.00€]
[Lenovo ThinkPad X1 Carbon, CPU: i7-1260P, RAM: 16GB, Price: 1899.00€]
[Apple MacBook Pro 16, CPU: M2 Max, RAM: 32GB, Price: 3200.00€]
[Razer Blade 15, CPU: i9-12900H, RAM: 32GB, Price: 3600.00€]

4.4 Is there a cheap laptop (<400€)?
Answer: false

4.5 All models as one String
MacBook Air M2, XPS 13, ThinkPad X1 Carbon, ZenBook 14, Pavilion 15, MacBook Pro 16, Inspiron 15, Blade 15, IdeaPad 3
```

Καλή διασκέδαση και καλή επιτυχία!